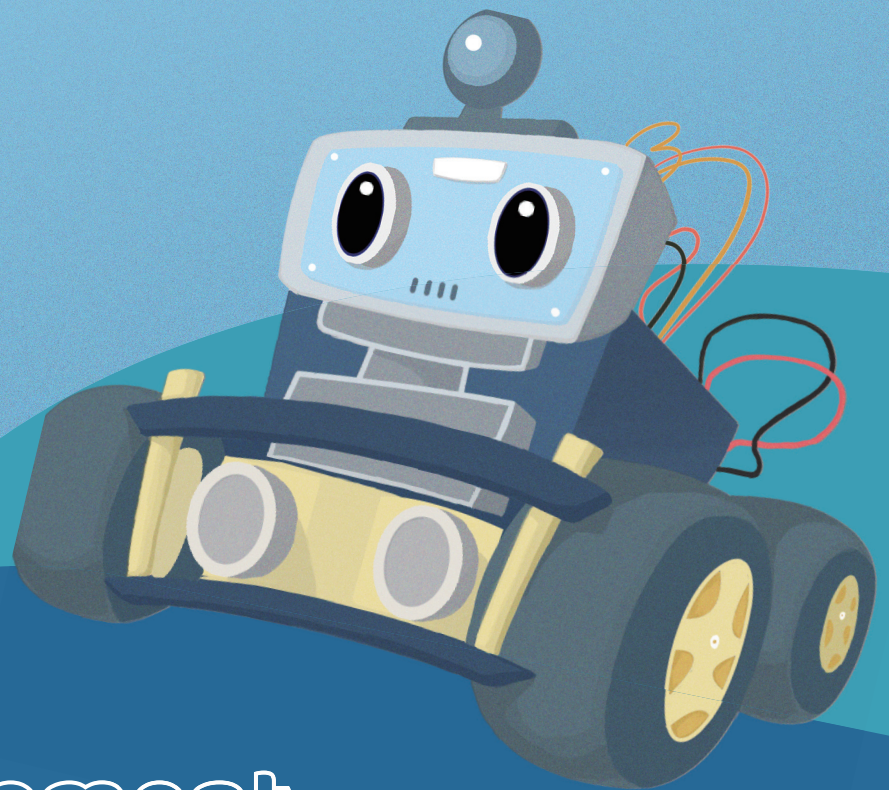


2

SMART ROBOT CAR V4.0 WITH CAMERA



Movement





Introduction:

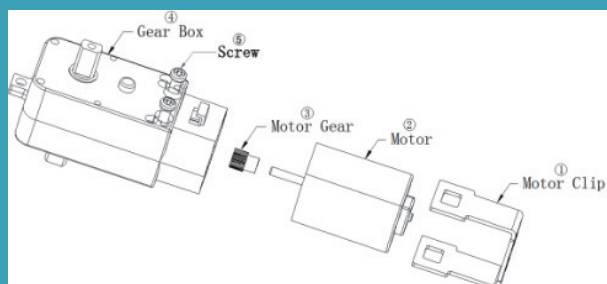
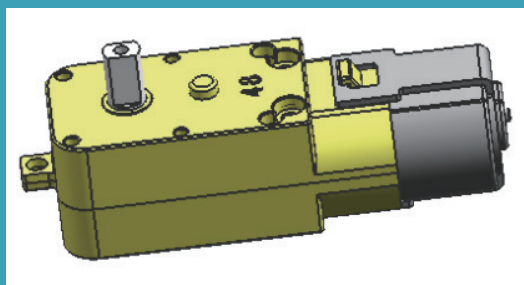
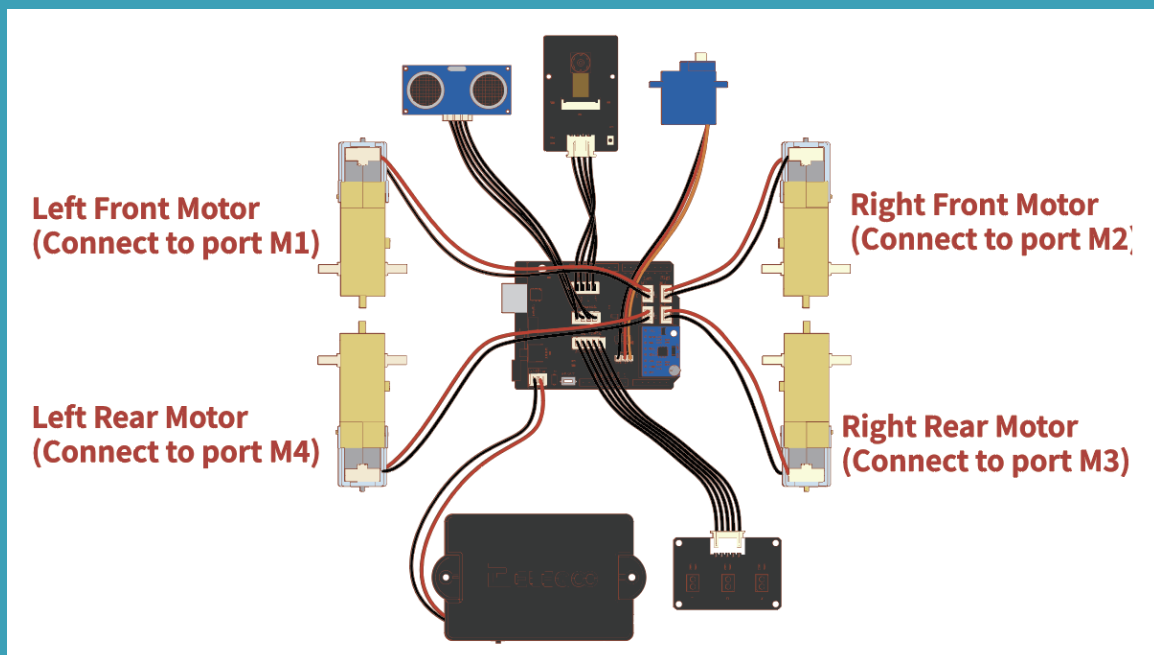
- + In this lesson, we will teach you how to make the car move as you wish by controlling the motor.



Material Preparation:

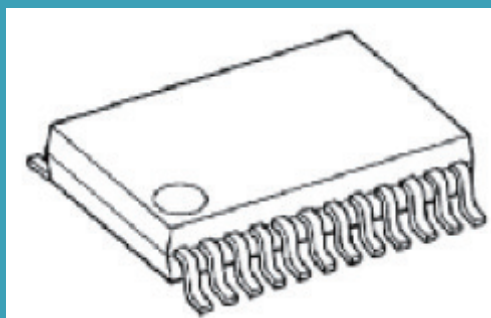
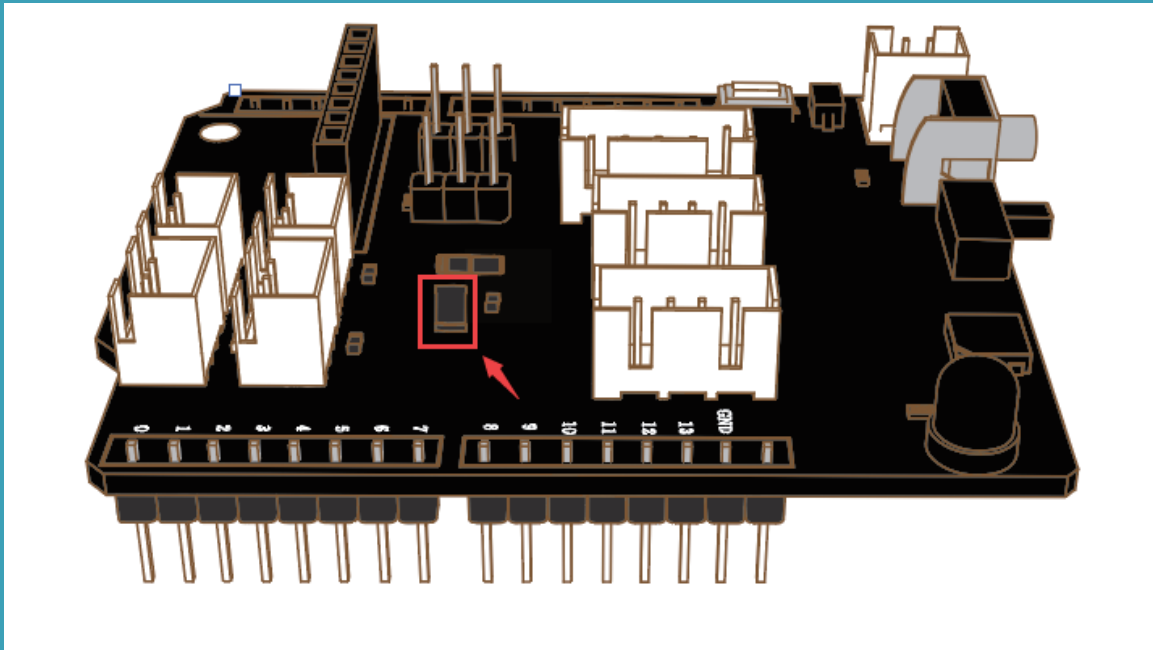
- + A Smart Robot Car (with battery)
A USB Cable

- + First of all, let's take a look at the 4 motors we used in our kit.



+ DC motor is the inductive load of large current (load with inductance parameters), while the IO port of Arduino UNO single-chip microcomputer we used has weak load capacity (**output current capacity**), and the driving capacity is not enough to supply enough current for the motor to rotate.

Therefore, we chose to use the motor driver chip TB6612. (**Although there may be slight differences in the performance of different driver chips, they are generally the same in usage.**):



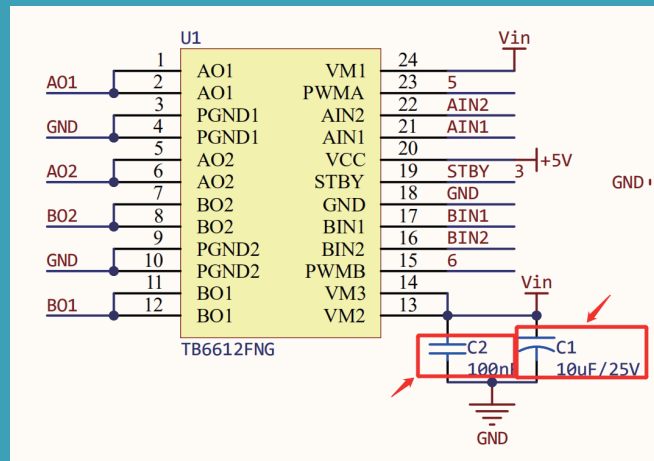
+ The TB6612 is a DC motor driver device with a high-current MOSFET-H bridge structure and dual-channel output to drive two motors simultaneously. It has two advantages:

1. TB6612 supplies enough current to the motor. Each H bridge of the TB6612 can provide up to 1.2A of output current. It operates within the motor power voltage range of 0V to 15V and the device power voltage range of 2V to 5.5V.

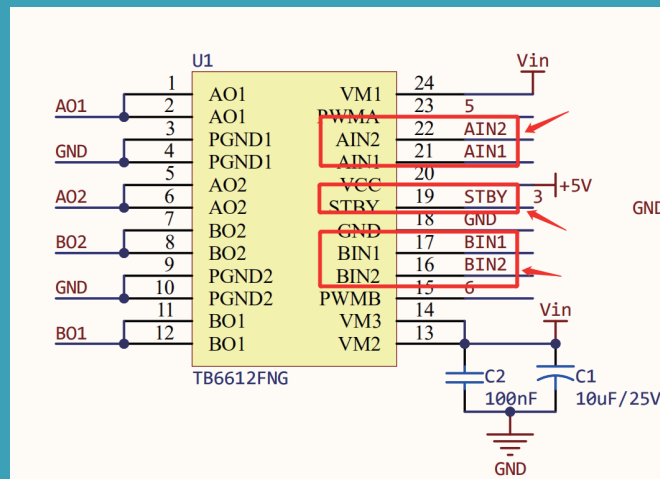
2. TB6612 acts as isolation to avoid damage of control devices by the impulse current generated by the motor.

✚ Perhaps everyone is more familiar with L298N. In fact, the use of the two is basically the same. Compared with L298N's heat dissipation and peripheral diode freewheeling circuit, it does not require an external heat sink, and the peripheral circuit is simple. It only needs an external power filter capacitor. Directly driving the motor is beneficial to reduce the size of the system, and the frequency of up to 100KHZ is sufficient to meet most of our needs.

Please open the folder of last level for details: [Related chip information -> TB6612 and SmartCar-Shield-V1.1](#)



Moreover, the pins of TB6612 connected to UNO can also be seen in this diagram.



EN stands for enable pin, which changes the motor speed by changing PWM (0~255).

PH stands for phase pin, and the rotation direction (positive/reverse) of the motor is changed by changing the pin level (0/1).

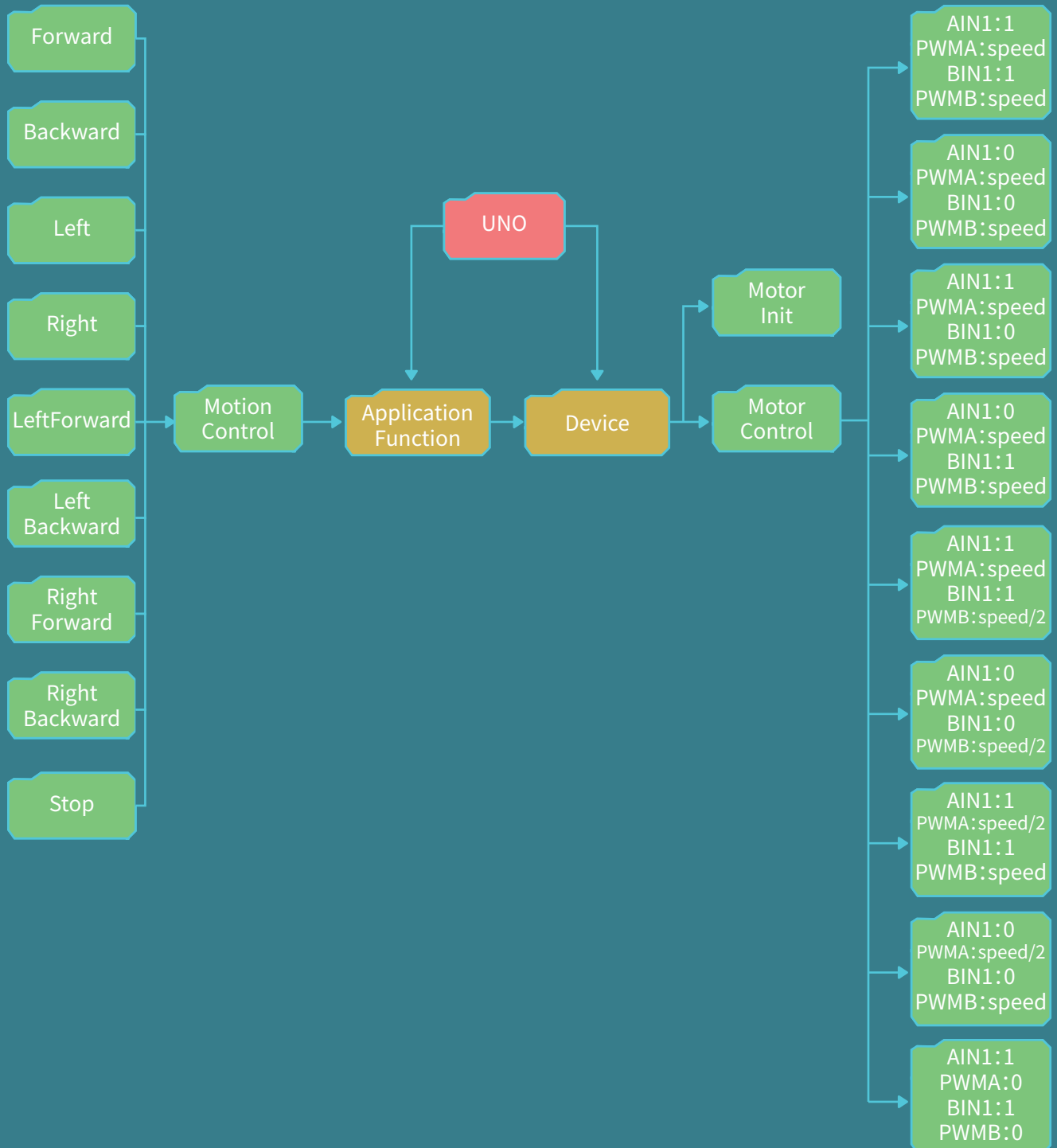
In motor A, PWMA is connected to the UNO~D5 pins and AIN1 is connected to the UNO **D7** pin.

In motor B, PWMB is connected to the UNO~D6 pins and BIN1 is connected to the UNO **D8** pin.

Finally, the driving method can be obtained from [Related chip information -> TB6612](#).

+ Then, we can start writing program according to the flow chart below.
Please open **Demo1** in the current folder:

Implementation principle of Move



+ First of all, we can see that there are four files.

1. Demo1 2. ApplicationFunctionSet_000.cpp 3. DeviceDriverSet_000.cpp 4. DeviceDriverSet_000.h

+ Next, let's take a look at the definition of the relevant pins and variables:

```
C: // in DeviceDriverSet_xxx0.h

/*Motor*/
class DeviceDriverSet_Motor
{
public:
    void DeviceDriverSet_Motor_Init(void);
    #if _Test_DeviceDriverSet
        void DeviceDriverSet_Motor_Test(void);
    #endif
    void DeviceDriverSet_Motor_control(boolean direction_A, uint8_t speed_A,
                                      boolean direction_B, uint8_t speed_B,
                                      boolean controlled
    );
private:

    #define PIN_Motor_PWMA 5
    #define PIN_Motor_PWMB 6
    #define PIN_Motor_BIN_1 8
    #define PIN_Motor_AIN_1 7
    #define PIN_Motor_STBY 3

    public:
    #define speed_Max 255
    #define direction_just true
    #define direction_back false
    #define direction_void 3

    #define Duration_enable true
    #define Duration_disable false
    #define control_enable true
    #define control_disable false
};
```

+ Define the maximum speed of motor rotation:

```
C:/ // in DeviceDriverSet_xxx0.h

#define speed_Max 255
```

+ Defining flag variables

+ Motor rotation direction flag

```
C:/ // in DeviceDriverSet_xxx0.h

#define direction_just true
#define direction_back false
#define direction_void 3
```

+ Other functional flags

```
C:/ // in DeviceDriverSet_xxx0.h

#define Duration_enable true
#define Duration_disable false
#define control_enable true
#define control_disable false
```

+ These pins need to be initialized before they can be operated.

```
C:/ //in DeviceDriverSet_xxx0.cpp

extern DeviceDriverSet_Motor AppMotor;
void DeviceDriverSet_Motor::DeviceDriverSet_Motor_Init(void)
{
    pinMode(PIN_Motor_STBY, OUTPUT);
    pinMode(PIN_Motor_PWMA, OUTPUT);
    pinMode(PIN_Motor_PWMB, OUTPUT);
    pinMode(PIN_Motor_AIN_1, OUTPUT);
    pinMode(PIN_Motor_BIN_1, OUTPUT);
}
```

+ And call it in 'setup'.

```
C:/ DeviceDriverSet_Motor AppMotor;  
void setup()  
{  
  AppMotor.DeviceDriverSet_Motor_Init();  
}
```

+ After initializing the pins, we can change the status of the pins to control the motor and make the motor rotate as we want.

1. Car Motion

```
C:/ //in ApplicationFunctionSet_xxx0.cpp  
static void ApplicationFunctionSet_SmartRobotCarMotionControl  
(SmartRobotCarMotionControl direction, uint8_t is_speed)
```

parameter:

direction: Passing motion direction control sequence.

is_speed: 0~255

```
C:/ //in ApplicationFunctionSet_xxx0.cpp  
  
/*Motion direction control sequence*/  
enum SmartRobotCarMotionControl  
{  
  Forward,    //(1)  
  Backward,   //(2)  
  Left,       //(3)  
  Right,      //(4)  
  LeftForward, //(5)  
  LeftBackward, //(6)  
  RightForward, //(7)  
  RightBackward, //(8)  
  stop_it    //(9)  
};
```

2. Motor control

```
C://in DeviceDriverSet_xxx0.cpp
void DeviceDriverSet_Motor::DeviceDriverSet_Motor_control
    (boolean direction_A, uint8_t speed_A, //motor A
     boolean direction_B, uint8_t speed_B, //motor B
     boolean controlled //AB enable true
    )
```

parameter:

direction_A,direction_B: Passing motor rotation direction flag.

speed_A,speed_B: range 0~255

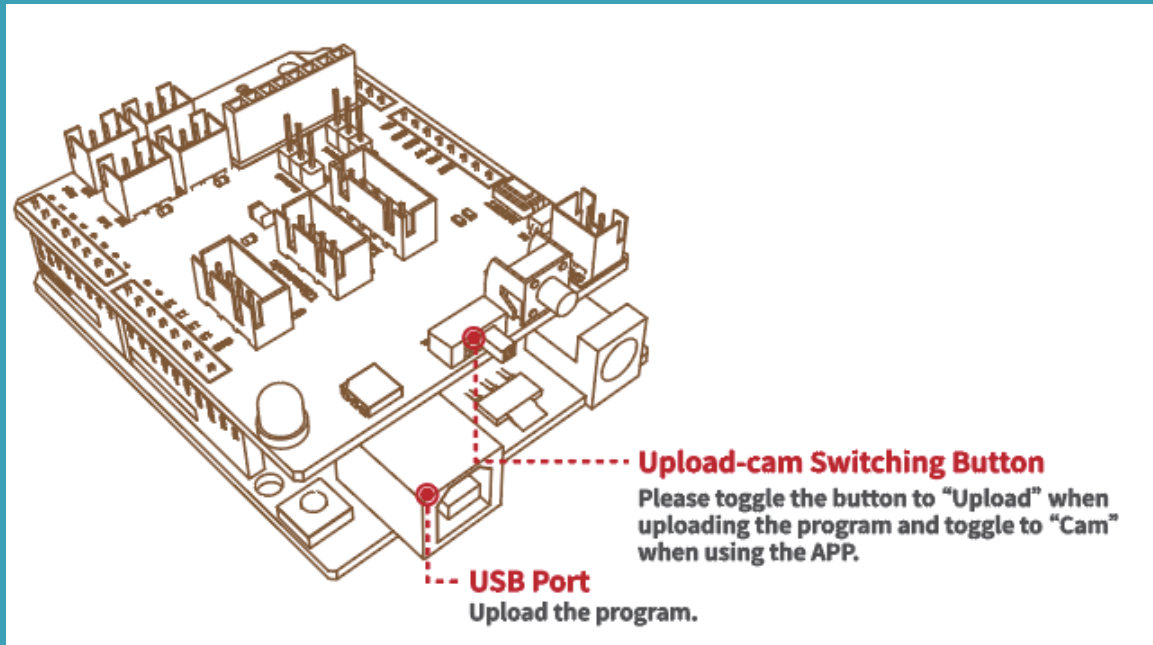
```
C:// in DeviceDriverSet_xxx0.h

#define direction_just true //motor forward
#define direction_back false //motor backward
#define direction_void 3
```

+ In the end, make the car move in each direction for one second after 2s waiting.

```
C:// void setup() {
    AppMotor.DeviceDriverSet_Motor_Init();
    delay(2000);
    for (Application_SmartRobotCarxxx0.Motion_Control = 0;
         Application_SmartRobotCarxxx0.Motion_Control < 9;
         Application_SmartRobotCarxxx0.Motion_Control =
             Application_SmartRobotCarxxx0.Motion_Control + 1)
    {
        ApplicationFunctionSet_SmartRobotCarMotionControl
        (Application_SmartRobotCarxxx0.Motion_Control /*direction*/,
         255 /*speed*/);
        delay(1000);
    }
}
```

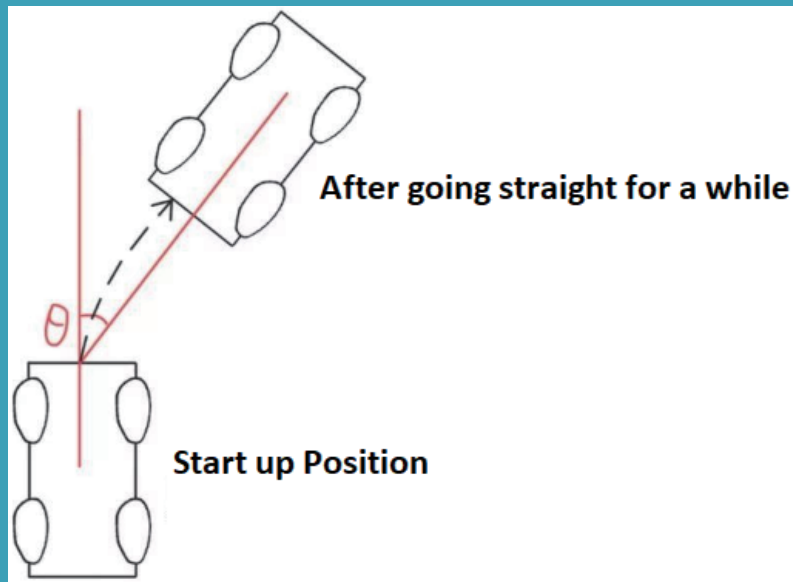
+ Upload the program. (Please toggle the “Upload-Cam” button to “Upload” when uploading the program.) Place the car on the ground and turn on the switch after the program has been uploaded successfully. Then, you will see the car move forward, move backward, turn left, turn right, turn front-left, left-rear, turn front-right, turn right-rear at 1s intervals and finally stop.



After **Demo1** was executed, if you are careful enough, you may notice that there are some deviations in the direction of the Smart Robot Car when it moves forward and backward. On the one hand, different terrain has different friction, on the other hand, even the same type of motor may have slight differences in hardware characteristics and slowdown structure, as well as disturbances during operation, such as momentary wheel slippage, tiny obstacles and other factors that may cause the left and right wheels to rotate at different speeds when going "straight". That's why it deviates from the path slightly.

You can upload the program of **Demo2** to keep observing if you don't see it clearly in the Demo1.

Upload the program. (Please toggle the button B/U to “Upload” when uploading the program.) You will see the car move forward and backward at 1s of intervals. In the process of moving forward and backward, the moving route will have a more obvious deviation.



✚ From the above, we know that similar to driving a car, if you close your eyes, it is difficult to keep walking straight for a long distance even if you straighten the steering wheel. Therefore, we have to get the deviation by eye and then correct it by turning the steering wheel manually to walk a straight line. **Therefore, if we want the car to go in a straight line, we must introduce closed-loop control.**

Closed-loop control is a basic concept of cybernetics. Refers to a control relationship in which the controlled output returns to the controlled input in a certain way and exerts a control influence on the input.

The advantage of closed-loop control over open-loop control is that it allows the results of the control to be fed back and compared to the desired values and adjusts the control effect based on their errors.

✚ Therefore, by applying closed-loop control to Smart Robot Car, we can perform speed corrections via the PID algorithm after integrating the yaw axis data by obtaining the angular velocity filter of the deviation via the QMI8658C. Next, please open **Demo3** in the current folder.

✚ In this routine, we first ported the "**QMI8658C.h**" library.

MPU6050.cpp

MPU6050.h

+ Let's first define a related class and class variable.

C:/

```
//in QMI8658C.h
#ifndef _QMI8658C_H_

#define _QMI8658C_H_
#include <Arduino.h>
class QMI8658C
{
public:
    bool QMI8658C_dveInit(void);
    bool QMI8658C_calibration(void);
    bool QMI8658C_dveGetEulerAngles(float *gyro, float *yaw);
    bool QMI8658C_dveGetEulerAngles(float *yaw);
    void QMI8658C_Check(void);

public:
    int16_t ax, ay, az, gx, gy, gz;
    float pith, roll, yaw;
    unsigned long now, lastTime = 0;
    float dt;    //Derivative time
    float agz = 0; //Angle variable
    long gzo = 0; //Gyro offset
};

//extern QMI8658C _QMI8658C;
#endif
```

+ Initialize the QMI8658C and ensure that the drive is succeed.

```
C:/ //in QMI8658C.cpp

bool QMI8658C::QMI8658C_dvelInit(void)
{
    Wire.begin();
    uint8_t chip_id = 0x00;
    uint8_t cout;
    do
    {
        I2Cdev::readBytes(address, WHO_AM_I, 1, &chip_id);
        Serial.print("WHO_AM_I: 0X");
        Serial.println(chip_id, HEX);
        cout += 1;
        if (cout > 10)
        {
            return true;
        }
        delay(10);
    } while (chip_id == 0); //Ensure that the slave device is online
    (Wait forcibly to get the ID)

    I2Cdev::writeByte(address, CTRL1, 0x40);
    //Serial Interface and Sensor Enable
    I2Cdev::writeByte(address, CTRL7, 0x03);
    //Enable Sensors and Configure Data Reads
    <Enable Gyroscope Accelerometer>

    I2Cdev::writeByte(address, CTRL2, 0x04);
    //Accelerometer Settings< $\pm 2g$  500Hz>
    I2Cdev::writeByte(address, CTRL3, 0x64);
    //Gyroscope Settings<  $\pm 2048dps$  500Hz>
    I2Cdev::writeByte(address, CTRL5, 0x11);
    //Sensor Data Processing Settings
    <Enable Gyroscope Accelerometer Low pass filter>

    delay(1000);

    // unsigned short times = 1000; //Sampling times
    // for (int i = 0; i < times; i++)
    // {
    //     gz = QMI8658C_readBytes();
    //     gzo += gz;
    // }
    // gzo /= times; //Calculate gyroscope offset

    QMI8658C_calibration();
    return false;
}
```

+ Because of the data drift problem of QMI8658C, we had better use mean filtering when sampling, and then obtain the z-axis angular velocity value when the car is first placed.

```
C://in QMI8658C.cpp

bool QMI8658C::QMI8658C_calibration(void)
{
    unsigned short times = 1000; //Sampling times
    for (int i = 0; i < times; i++)
    {
        gz = QMI8658C_readBytes();
        gzo += gz;
    }
    gzo /= times; //Calculate gyroscope offset

    return false;
}
```

+ Then, put it into the `setup()` for call.

```
C:// void setup() {
    Serial.begin(9600);
    AppQMI8658C.QMI8658C_dveInit();
    delay(2000);
    AppQMI8658C.QMI8658C_calibration();
}
```

✚ From QMI8658C, the sensitivity factor is known to be 32 LSB (count) /°/s. After obtaining the original sensor data, we can calculate the angular velocity by dividing the original sensor data by their sensitivity scaling factor:

angular velocity along the Z-axis = (gyro z-axis raw data /32) °/s.

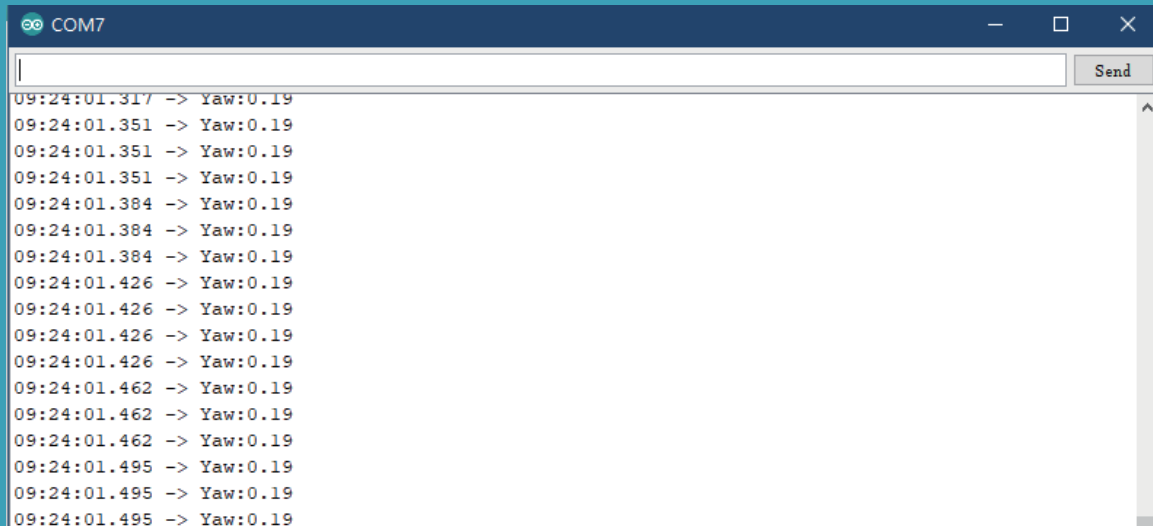
Because the execution time of the program is less than 1S, we need to multiply the final result by time.

Finally, the yaw value can be obtained by integrating the resulting angular velocity.

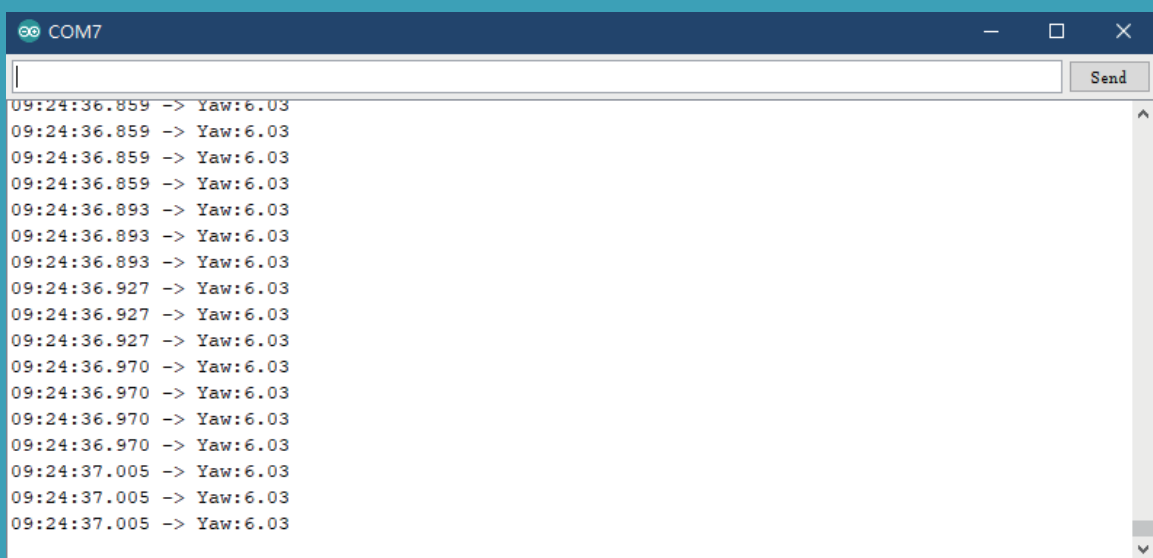
C:/ //in QMI8658C.cpp

```
bool QMI8658C::QMI8658C_dveGetEulerAngles(float *Yaw)
{
    unsigned long now = millis();
    dt = (now - lastTime) / 1000.0;
    lastTime = now;
    gz = QMI8658C_readBytes();
    float gyroz = -(gz - gzo) / 32.00 * dt;
    if (fabs(gyroz) <= 0.05)
    {
        gyroz = 0.00;
    }
    agz += gyroz;
    *Yaw = agz;
    *gyro = gyroz;
    return false;
}
```

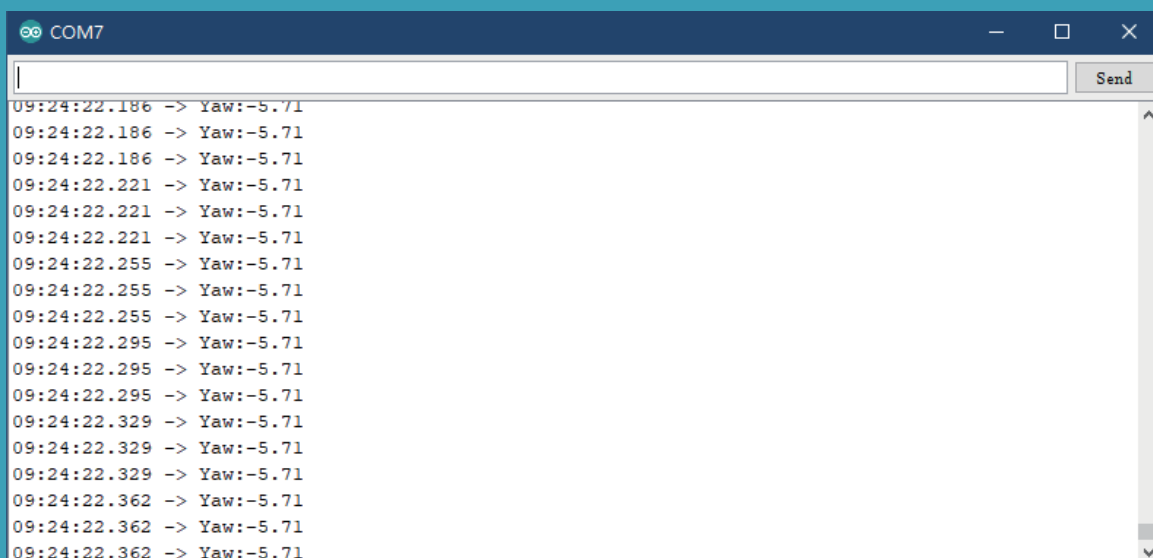
✚ Please place the car steadily before uploading the program. (Please toggle the “Upload-Cam” button to “Upload” when uploading the program.) After uploading the program, if you do not move the car, wait two seconds and open the serial port, the yaw angle will fluctuate around 0. If you move the car slightly, the yaw angle will also change accordingly. The straight-line walking function of the car is programmed based on this principle.



```
COM7
09:24:01.317 -> Yaw:0.19
09:24:01.351 -> Yaw:0.19
09:24:01.351 -> Yaw:0.19
09:24:01.351 -> Yaw:0.19
09:24:01.384 -> Yaw:0.19
09:24:01.384 -> Yaw:0.19
09:24:01.384 -> Yaw:0.19
09:24:01.426 -> Yaw:0.19
09:24:01.426 -> Yaw:0.19
09:24:01.426 -> Yaw:0.19
09:24:01.426 -> Yaw:0.19
09:24:01.462 -> Yaw:0.19
09:24:01.462 -> Yaw:0.19
09:24:01.462 -> Yaw:0.19
09:24:01.495 -> Yaw:0.19
09:24:01.495 -> Yaw:0.19
09:24:01.495 -> Yaw:0.19
```



```
COM7
09:24:36.859 -> Yaw:6.03
09:24:36.859 -> Yaw:6.03
09:24:36.859 -> Yaw:6.03
09:24:36.859 -> Yaw:6.03
09:24:36.893 -> Yaw:6.03
09:24:36.893 -> Yaw:6.03
09:24:36.893 -> Yaw:6.03
09:24:36.927 -> Yaw:6.03
09:24:36.927 -> Yaw:6.03
09:24:36.927 -> Yaw:6.03
09:24:36.970 -> Yaw:6.03
09:24:36.970 -> Yaw:6.03
09:24:36.970 -> Yaw:6.03
09:24:36.970 -> Yaw:6.03
09:24:37.005 -> Yaw:6.03
09:24:37.005 -> Yaw:6.03
09:24:37.005 -> Yaw:6.03
```



```
COM7
09:24:22.186 -> Yaw:-5.71
09:24:22.186 -> Yaw:-5.71
09:24:22.186 -> Yaw:-5.71
09:24:22.221 -> Yaw:-5.71
09:24:22.221 -> Yaw:-5.71
09:24:22.221 -> Yaw:-5.71
09:24:22.255 -> Yaw:-5.71
09:24:22.255 -> Yaw:-5.71
09:24:22.255 -> Yaw:-5.71
09:24:22.295 -> Yaw:-5.71
09:24:22.295 -> Yaw:-5.71
09:24:22.295 -> Yaw:-5.71
09:24:22.329 -> Yaw:-5.71
09:24:22.329 -> Yaw:-5.71
09:24:22.329 -> Yaw:-5.71
09:24:22.362 -> Yaw:-5.71
09:24:22.362 -> Yaw:-5.71
09:24:22.362 -> Yaw:-5.71
```

+ After the yaw data is obtained, the closed-loop feedback control can be achieved through PID algorithm to adjust the wheel speed so as to achieve straight-line walking. Next, please open **Demo4** in the current folder.

KP scale parameter is the best data we can get through repeated debugging.

UpperLimit is the maximum speed of the car.

```
C:/ //in ApplicationFunctionSet_xxx0.cpp

static void ApplicationFunctionSet_SmartRobotCarMotionControl
(SmartRobotCarMotionControl direction, uint8_t is_speed)
{
    Kp = 10;
    UpperLimit = 255;
}
```

+ The yaw data is acquired at the first startup, and the data is updated every ten millimeters.

```
C:/ //in ApplicationFunctionSet_xxx0.cpp

Static void ApplicationFunctionSet_SmartRobotCarLinearMotionControl
(SmartRobotCarMotionControl direction, uint8_t directionRecord,
uint8_t speed, uint8_t Kp, uint8_t UpperLimit)
{
    ....
    if (en != directionRecord || millis() - is_time > 10)
    {
        AppMotor.DeviceDriverSet_Motor_control(/*direction_A*/
direction_void, /*speed_A*/ 0,
/*direction_B*/ direction_void, /*speed_B*/ 0, /*controlled*/
control_enable); //Motor control
        AppQMI8658C.QMI8658C_dveGetEulerAngles(&Yaw);
        is_time = millis();
    }
    .....
}
```

+ Then, we need to perform P proportional control in the PID algorithm on the acquired input data.

```
C:/ //in ApplicationFunctionSet_xxx0.cpp
```

```
static void ApplicationFunctionSet_SmartRobotCarLinearMotionControl
(SmartRobotCarMotionControl direction, uint8_t directionRecord,
uint8_t speed, uint8_t Kp, uint8_t UpperLimit)
{
    ....
    if(en != directionRecord )
    {
        en = directionRecord;
        yaw_So = Yaw;
    }
    .....
}
```

Then, we need to perform P proportional control in the PID algorithm on the acquired input data.

The principle of P control is roughly as follows: Suppose you are running to an end point, when you are far away from the end point, you will run at full speed; when you are close to the end point, you will slow down; and when you are about to reach the end point, you will also about to stop.

+ And then, remember to limit the corrected speed so that it does not exceed the maximum speed.

```
C:/ //in ApplicationFunctionSet_xxx0.cpp

static void ApplicationFunctionSet_SmartRobotCarLinearMotionControl
(SmartRobotCarMotionControl direction, uint8_t directionRecord,
 uint8_t speed, uint8_t Kp, uint8_t UpperLimit)
{
    .....
    int R = (Yaw - yaw_So) * Kp + speed;
    if (R > UpperLimit)
    {
        R = UpperLimit;
    }
    else if (R < 10)
    {
        R = 10;
    }
    int L = (yaw_So - Yaw) * Kp + speed;
    if (L > UpperLimit)
    {
        L = UpperLimit;
    }
    else if (L < 10)
    {
        L = 10;
    }
    .....
}
```

Finally, upload the program. (Please toggle the “Upload-Cam” button to “Upload” when uploading the program.) After you have uploaded the program, please place the car on the ground steadily first and then switch it on. Wait for two seconds, you will find that the car will repeatedly move forward for 3 seconds and then move backward for 3 seconds in a relatively straight line.